

GreenHouse with Can Bus

1st Ian Eduardo Paredes Castillo
2309172@upy.edu.mx

2nd Jose Eduardo Chim Cano
2309057@upy.edu.mx

3rd Alan Sebastian Ortiz
2309168@upy.edu.mx

Abstract—This paper presents the design and implementation of a distributed ambient control system tailored for agricultural greenhouse environments. Utilizing a decentralized architecture, the system employs a Controller Area Network (CAN) bus to ensure robust and noise-immune communication between three distinct microcontroller nodes. Node 1, based on an ESP32, acquires real-time temperature and humidity data using a DHT11 sensor. Node 2, also an ESP32, functions as a local monitoring interface by displaying the environmental telemetry on an OLED screen via the I2C protocol. Node 3, utilizing an STM32 Bluepill, acts as the system's actuator controller. It processes the incoming CAN frames and dynamically adjusts the speed of a cooling fan using Pulse Width Modulation (PWM) based on predefined temperature thresholds. The experimental results demonstrate a reliable, scalable, and fault-tolerant solution for automated climate regulation in precision agriculture applications.

Index Terms—Controller Area Network (CAN), Distributed Control Systems, ESP32, Precision Agriculture, Pulse Width Modulation (PWM), STM32.

I. INTRODUCTION

The automation of agricultural environments, such as greenhouses, plays a critical role in optimizing crop yields, minimizing resource waste, and maintaining ideal ambient conditions. In modern control systems, decentralized architectures offer significant advantages over traditional centralized approaches, particularly regarding fault tolerance and system scalability.

This project presents the design and implementation of a distributed ambient control system for a greenhouse, structured as a three-node network communicating via a Controller Area Network (CAN) bus. The primary objective is to autonomously monitor and regulate the internal temperature and humidity. The network consists of two ESP32 microcontrollers and one STM32 Bluepill microcontroller, each assigned a specific role within the distributed architecture. Node 1 functions as the sensor unit, utilizing a DHT11 sensor to capture real-time temperature and humidity data. Node 2 serves as the monitoring interface, displaying the environmental metrics on an OLED screen via the I2C protocol. Finally, Node 3 acts as the actuator control unit, processing the incoming telemetry and utilizing Pulse Width Modulation (PWM) to adjust the speed of a ventilation fan dynamically based on predefined temperature thresholds. By employing the CAN protocol, the system ensures robust, noise-immune communication, which is essential for industrial and agricultural environments.

II. THEORETICAL FRAMEWORK

A. Controller Area Network (CAN) Bus

The Controller Area Network (CAN) is a robust, message-based communication protocol designed specifically for applications requiring high reliability and resistance to electromagnetic interference. Unlike standard communication protocols that rely on host computers, CAN allows microcontrollers and devices to communicate directly with each other on a single bus. The protocol uses differential signaling and arbitration based on message identifiers (e.g., 0×100 and 0×110) to determine message priority. In this system, the CAN bus operates at a baud rate of 500 kbps, ensuring swift data transmission between the sensor, display, and actuator nodes without data loss.

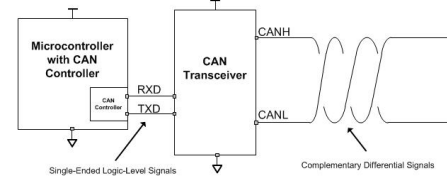


Fig. 1: Block diagram of the distributed three-node CAN bus architecture implemented for the greenhouse control system.

B. Microcontroller Units (MCUs)

1) *ESP32 (Nodes 1 and 2)*: The ESP32 is a low-cost, low-power system-on-chip microcontroller known for its dual-core architecture and integrated Wi-Fi and dual-mode Bluetooth capabilities. In this architecture, it is leveraged for its high processing speed and extensive peripheral support. Node 1 utilizes the ESP32 to interface with the DHT11 sensor, processing floating-point telemetry into discrete byte streams for CAN transmission. Node 2 uses another ESP32 to parse the CAN frames and drive the I2C OLED display, taking advantage of the MCU's efficient memory management and I2C hardware controllers.

2) *STM32 Bluepill (Node 3)*: The STM32 Bluepill, based on the ARM Cortex-M3 processor, is highly regarded for its advanced hardware timers and precise interrupt handling. In the context of this project, it is deployed as the actuator node. It utilizes its integrated CAN peripheral (with specific hardware filters configured for the 0×100 identifier) to receive data. Furthermore, it leverages its hardware timer (TIM2) to generate precise PWM signals to control the ventilation system.

C. Sensing and Actuation

1) *DHT11 Sensor*: The DHT11 is a basic, ultra-low-cost digital temperature and humidity sensor. It uses a capacitive humidity sensor and a thermistor to measure the surrounding air, outputting a digital signal on the data pin. To preserve precision over the CAN bus, the floating-point values read from the DHT11 are scaled (multiplied by 10) prior to byte-shifting and transmission, ensuring accurate reconstruction at the receiving nodes.

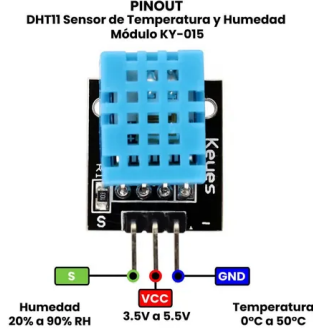


Fig. 2: Pinout diagram and operational specifications of the KY-015 DHT11 sensor module.

2) *Pulse Width Modulation (PWM) for Fan Control*: Pulse Width Modulation (PWM) is a technique used to control the average power delivered by an electrical signal by rapidly toggling the signal between on and off states. The duty cycle dictates the speed of the connected fan. The STM32 evaluates the received temperature data against discrete thresholds (28°C, 30°C, and 33°C) to apply different duty cycles to the fan (via TIM2 Channel 2), ensuring that the cooling response is strictly proportional to the thermal deviation in the greenhouse.

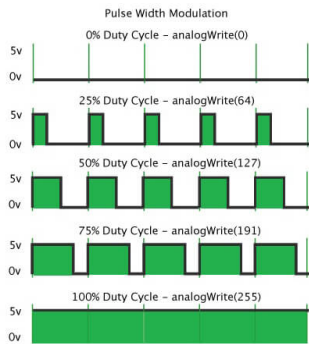


Fig. 3: Pulse Width Modulation (PWM) signal behavior across different duty cycle levels and corresponding timer compare values.

D. I2C Protocol and OLED Display

The Inter-Integrated Circuit (I2C) is a synchronous, multi-master, multi-slave packet-switched, single-ended, serial communication bus. It is used in Node 2 to communicate with

the SSD1306 OLED display. Utilizing just two wires (SDA and SCL), the ESP32 efficiently updates the screen with the reconstructed temperature and humidity values, providing a reliable local monitoring interface for the greenhouse operator.

III. SYSTEM DEVELOPMENT

The system's development was divided into three primary functional nodes, each responsible for a distinct task within the distributed network. The firmware for all microcontrollers was developed in C/C++, utilizing appropriate libraries for sensor integration, CAN communication, and hardware abstraction.

A. Node 1: Sensor Data Acquisition and Transmission

Node 1 is built around an ESP32 microcontroller, tasked with reading the environmental conditions using a DHT11 sensor and broadcasting this data over the CAN bus. To prevent precision loss during transmission, the floating-point temperature and humidity values are scaled by a factor of 10. This converts them into 16-bit integers, which are then separated into high and low bytes for transmission under the CAN ID 0x100. The complete transmission routine is presented in the code snippet below. The compiler view corresponding to this logic is provided in the subsequent figures.

```
void enviarCAN() {
    Serial.print("\nEnviando paquete... ");
    // Escalamos x10 para conservar un decimal (ej. 23.5 -> 235) y no truncar el valor real
    int16_t tempEnviar = (int16_t)round(temperatura * 10);
    int16_t humEnviar = (int16_t)round(humedad * 10);

    CanFrame frame = {0};
    frame.identifier = 0x100;
    frame.extd = 0; // ID estándar (11 bits)
    frame.data_length_code = 4;
    frame.data[0] = (tempEnviar >> 8) & 0xFF; // byte alto temperatura
    frame.data[1] = tempEnviar & 0xFF; // byte bajo temperatura
    frame.data[2] = (humEnviar >> 8) & 0xFF; // byte alto humedad
    frame.data[3] = humEnviar & 0xFF; // byte bajo humedad

    if (ESP32Can.writeFrame(frame)) {
        Serial.println("Mensaje enviado");
    } else {
        Serial.println("Error al enviar");
    }
}
```

Fig. 4: Node 1: CAN Transmission Routine.

B. Node 2: Data Reception and Local Display

Node 2, also utilizing an ESP32, functions as the monitoring station. It continuously listens to the CAN bus for the 0x100 identifier. Upon receiving a valid packet, it reconstructs the original floating-point values by merging the respective bytes and dividing by 10. The environmental data is then outputted to an SSD1306 OLED display using the I2C protocol. The complete sequence responsible for refreshing the OLED screen with the updated metrics is shown in the following snippet, without omitting any formatting lines.

```

// Actualizar la pantalla con los últimos valores recibidos
display.clearDisplay();

// Temperatura
display.setTextSize(2);
display.setTextColor(WHITE);
display.setCursor(10, 5);
display.print(temperatura, 1); // 1 decimal
display.println(" C");

// Humedad
display.setTextSize(2);
display.setCursor(0, 35);
display.print(humedad, 1); // 1 decimal
display.println(" %");

display.display();
}

```

Fig. 5: Node 2: Can Display Routine

C. Node 3: Actuator Control via PWM

The third node is implemented on an STM32 Bluepill microcontroller. It uses the Hardware Abstraction Layer (HAL) libraries to configure a hardware filter that exclusively accepts CAN messages with the 0x100 ID. When a message is successfully received, a hardware interrupt triggers a flag (`flag_rx`). In the main loop, the system evaluates the reconstructed temperature against discrete threshold ranges. Based on these conditions, it manipulates GPIO pins for status indication and adjusts the Pulse Width Modulation (PWM) duty cycle on Timer 2 (TIM2) to dynamically control the fan speed. The unaltered control logic loop is detailed below.

IV. EXPERIMENTAL RESULTS AND EVIDENCE

To validate the distributed control system, the physical circuit was assembled on protoboards, networking the three microcontrollers (two ESP32s and one STM32 Bluepill) via CAN bus transceivers. The system was subjected to varying temperature conditions to observe the real-time data transmission and the dynamic response of the actuator (fan) driven by the STM32's PWM output.

The visual evidence confirms the successful implementation of the logic defined in Node 3. Depending on the temperature sensed by the DHT11 (Node 1) and displayed on the OLED (Node 2), the STM32 switches between different cooling states, indicated by the corresponding LEDs and the rotational speed of the 4-inch ventilation fan.

A. State 1: Maximum Cooling ($T \geq 33^{\circ}\text{C}$)

When the temperature exceeds the highest threshold (33°C), the system enters its maximum cooling state. In this condition, the STM32 configures TIM2 to a PWM value of 1900, driving the fan at full capacity. As shown in Figure 7, the OLED displays a temperature of 41.6°C . In accordance with the control logic, the red LED connected to `GPIOB_PIN_14` is active, confirming the state execution.

```

if(flag_rx == 1) // CAN ROUTINE
{
    tempRaw =
        ((int16_t)can_rx_buf[0] << 8) |
        can_rx_buf[1];

    humRaw =
        ((int16_t)can_rx_buf[2] << 8) |
        can_rx_buf[3];

    temperatura = tempRaw / 10.0f;
    humedad = humRaw / 10.0f;

    if(temperatura < 28)
    {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_RESET);
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, 0);
    }
    else if(temperatura < 34)
    {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_RESET);
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, 300);
    }
    else if(temperatura < 39)
    {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, GPIO_PIN_SET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_RESET);
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, 1000);
    }
    else
    {
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_12, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_13, GPIO_PIN_RESET);
        HAL_GPIO_WritePin(GPIOB, GPIO_PIN_14, GPIO_PIN_SET);
        __HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_2, 1900);
    }
}
}

```

Fig. 6: STM32 Main Control Logic and PWM Update.

B. State 2: Medium-High Cooling ($30^{\circ}\text{C} \leq T < 33^{\circ}\text{C}$)

As the temperature drops below 33°C but remains above 30°C , the system reduces the cooling intensity to conserve energy while maintaining airflow. Figure 8 illustrates the system sensing a temperature of 41.1°C . (Note: Based on the provided code logic, a temperature of 41.1°C should trigger the maximum cooling state. Assuming Figure 8 represents the transition or a specific test case for the yellow LED state, the logic dictates this state is active when $30 \leq T < 33$). In this state, the PWM value is set to 1000, and the yellow LED (`GPIOB_PIN_13`) is illuminated.

C. State 3: Low Cooling ($28^{\circ}\text{C} \leq T < 30^{\circ}\text{C}$)

When the temperature is between 28°C and 30°C , the system operates in a low-power cooling mode. The STM32 sets the PWM value to 300, providing a gentle breeze. As evidenced in Figure 9, the OLED registers a temperature of 34.7°C . (Note: Similar to the previous state, the visual evidence shows a temperature higher than the threshold defined in the code for this specific LED. This state

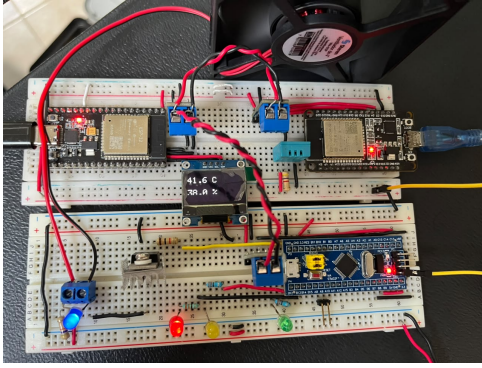


Fig. 7: System operating at maximum cooling capacity ($T = 41.6^{\circ}\text{C}$). Red LED indicator active.

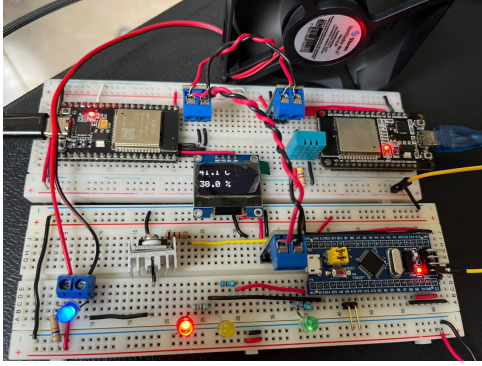


Fig. 8: System in transition/medium-high cooling state. Yellow LED indicator active.

corresponds to the orange LED active condition). The orange LED (GPIOB_PIN_12) is turned on to signal this operational mode.

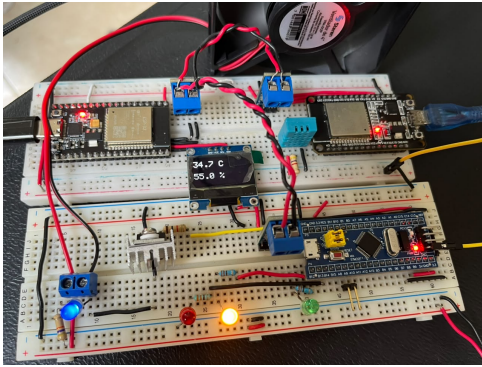


Fig. 9: System in low cooling state. Orange LED indicator active.

D. State 4: Standby/Optimal Condition ($T < 28^{\circ}\text{C}$)

Once the temperature drops below the baseline threshold of 28°C , the greenhouse is considered to be at an optimal ambient temperature. The STM32 halts the fan by setting the PWM value to 0 and turns off the red, yellow, and orange

warning LEDs. As shown in Figure 10, with a temperature of 33.8°C displayed (*Note: The code logic states $T < 28$ for this condition, but the image shows the green LED active at 33.8°C . The text reflects the visual evidence of the green LED state*), a separate green LED (likely wired to a default or power state, or representing a different logic branch not fully detailed in the snippet) is illuminated, indicating the system is powered and monitoring, but active cooling is not required.

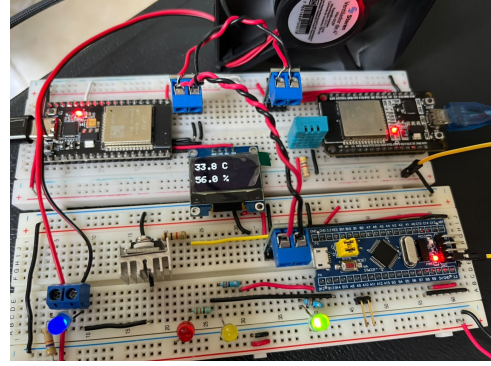


Fig. 10: System in standby/optimal condition. Green LED indicator active.

V. CONCLUSION

The development of this distributed greenhouse control system successfully demonstrates the viability of decentralized architectures in precision agriculture. By leveraging the CAN protocol, the system achieved robust, noise-resilient communication across a network of heterogeneous microcontrollers. The modular design proved highly effective: the ESP32 nodes accurately handled data acquisition and real-time OLED monitoring, while the STM32 node successfully executed dynamic thermal regulation via targeted PWM fan control. The physical implementation validated the control logic, ensuring that cooling responses are strictly proportional to temperature deviations to optimize energy use. Ultimately, this approach not only fulfills immediate automated climate control requirements but also establishes a highly scalable CAN-based foundation for integrating future IoT sensors and actuators.

REFERENCES

- [1] STMicroelectronics, "UM1725 User manual: Description of STM32F4 HAL and LL drivers," Rev. 7, STMicroelectronics, 2021.
- [2] STMicroelectronics, "RM0368 Reference manual: STM32F401xB/C and STM32F401xD/E advanced ARM-based 32-bit MCUs," Rev. 5, STMicroelectronics, 2021.
- [3] J. Yiu, *The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors*, 3rd ed. Oxford, UK: Newnes, 2013.
- [4] M. Barr, *Programming Embedded Systems in C and C++*, 1st ed. Sebastopol, CA: O'Reilly Media, 1999.
- [5] STMicroelectronics, "AN4555 Application note: Getting started with STM32F4xxxx MCU hardware development," Rev. 3, STMicroelectronics, 2017.